

Absolutely! This is the **third Git practical** and is commonly asked in FYIT practical exams.

Practical (c): Simulate a Merge Conflict and Resolve It

Aim

To create a merge conflict between two branches and resolve it successfully.

Prerequisites

You should have completed:

- ☒ Part (a): Created a Git repository
- ☒ Part (b): Created a feature branch and merged it

Your project should look like:

```
Git_Practical
├── hello.py
├── README.md
└── .git
```

Current `hello.py`:

```
print("Hello Git!")
print("Welcome to Git Repository")
```

Step 1: Open the Project

Open **VS Code**.

Open the project folder:

```
Git_Practical
```

Open the terminal:

```
Terminal → New Terminal
```

Step 2: Check the Current Branch

Type:

```
git branch
```

Output:

```
* main
```

You should be on the `main` branch.

Step 3: Create a New Branch

Create a branch named **feature-conflict**.

```
git checkout -b feature-conflict
```

Output:

```
Switched to a new branch 'feature-conflict'
```

Step 4: Modify `hello.py` in the Feature Branch

Open:

```
hello.py
```

Current code:

```
print("Hello Git!")  
print("Welcome to Git Repository")
```

Replace it with:

```
print("Hello Git!")  
print("Feature Branch Version")
```

Save (**Ctrl + S**).

Step 5: Commit the Changes

Check status:

```
git status
```

Stage the file:

```
git add hello.py
```

Commit:

```
git commit -m "Modified hello.py in feature branch"
```

Output:

```
1 file changed
```

Step 6: Switch Back to Main

```
git checkout main
```

Output:

```
Switched to branch 'main'
```

Step 7: Modify the Same Line in Main

Open:

```
hello.py
```

Current version (on main):

```
print("Hello Git!")  
print("Welcome to Git Repository")
```

Replace it with:

```
print("Hello Git!")  
print("Main Branch Version")
```

Save the file.

Step 8: Commit the Main Branch Changes

```
git add hello.py
```

Commit:

```
git commit -m "Modified hello.py in main branch"
```

Step 9: Merge the Feature Branch

Now merge:

```
git merge feature-conflict
```

Git will display:

```
Auto-merging hello.py
```

```
CONFLICT (content): Merge conflict in hello.py
```

```
Automatic merge failed; fix conflicts and then commit the result.
```



Congratulations! You have successfully created a merge conflict.

Step 10: Open the Conflicted File

Open:

```
hello.py
```

Git automatically inserts conflict markers.

You will see:

```
print("Hello Git!")
```

```
<<<<<<< HEAD
```

```
print("Main Branch Version")
```

```
=====
```

```
print("Feature Branch Version")
```

```
>>>>>>> feature-conflict
```

What Do These Symbols Mean?

<<<<<<< HEAD

Current branch (main)

=====

Separator

>>>>>>> feature-conflict

Code from the feature-conflict branch.

Step 11: Resolve the Conflict

Decide what you want to keep.

Option 1 (Keep Main)

```
print("Hello Git!")
print("Main Branch Version")
```

Option 2 (Keep Feature)

```
print("Hello Git!")
print("Feature Branch Version")
```

Option 3 (Recommended for Practical)

Keep **both**:

```
print("Hello Git!")
print("Main Branch Version")
print("Feature Branch Version")
```

Delete **all conflict markers** (<<<<<<<, =====, >>>>>>>).

Save the file.

Step 12: Stage the Resolved File

```
git add hello.py
```

Step 13: Complete the Merge

Commit the resolved merge:

```
git commit -m "Resolved merge conflict"
```

Output:

```
[main]  
Resolved merge conflict
```

Step 14: Verify the Merge

Type:

```
git log --oneline
```

Example:

```
abc123 Resolved merge conflict  
456xyz Modified hello.py in main branch  
789def Modified hello.py in feature branch
```

Step 15: Check Repository Status

```
git status
```

Output:

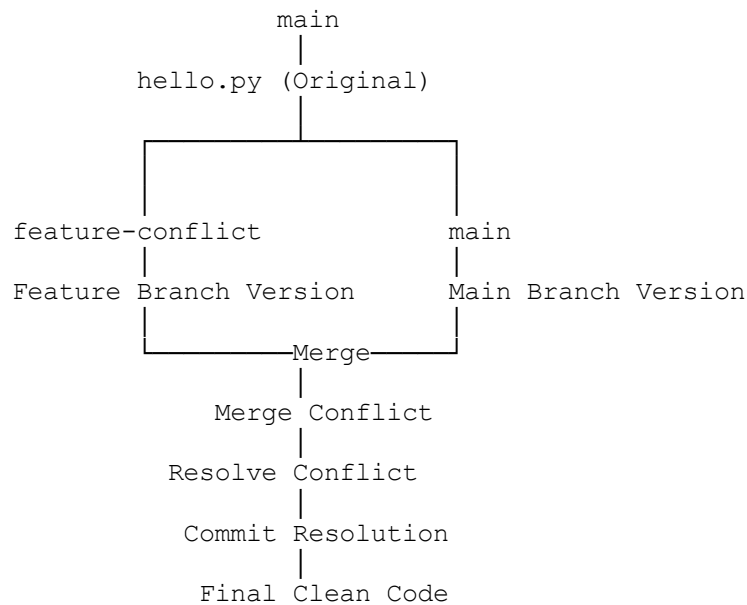
```
On branch main  
  
nothing to commit, working tree clean
```

This means the conflict has been resolved successfully.

Final `hello.py`

```
print("Hello Git!")
print("Main Branch Version")
print("Feature Branch Version")
```

Complete Workflow



Commands Summary

Step	Command	Purpose
1	<code>git checkout -b feature-conflict</code>	Create a new branch
2	<code>git add hello.py</code>	Stage changes
3	<code>git commit -m "Modified hello.py in feature branch"</code>	Commit feature changes
4	<code>git checkout main</code>	Switch to main
5	<code>git add hello.py</code>	Stage main changes
6	<code>git commit -m "Modified hello.py in main branch"</code>	Commit main changes
7	<code>git merge feature-conflict</code>	Attempt to merge
8	Edit <code>hello.py</code>	Resolve conflict manually
9	<code>git add hello.py</code>	Stage resolved file
10	<code>git commit -m "Resolved merge conflict"</code>	Finish the merge
11	<code>git status</code>	Verify repository is clean

Viva Questions

Q1. What is a merge conflict?

Answer: A merge conflict occurs when Git cannot automatically merge changes because the same part of a file has been modified differently in two branches.

Q2. Why does a merge conflict occur?

Answer: It occurs when two branches make different changes to the same line or nearby lines of the same file.

Q3. How do you resolve a merge conflict?

Answer: Open the conflicted file, remove the conflict markers, choose or combine the required changes, save the file, stage it using `git add`, and complete the merge with `git commit`.

Q4. What are conflict markers in Git?

Answer: Git inserts <<<<<<, =====, and >>>>>> into the file to show the conflicting changes from each branch.

Q5. Which command is used to merge branches?

Answer: The `git merge <branch-name>` command is used to merge one branch into the current branch.

This practical demonstrates the complete lifecycle of **creating a merge conflict and resolving it**, which is exactly what examiners typically expect in a Git practical.